

# ZK-Redact: Efficient Zero-Knowledge Authorized Redaction and Provenance Auditing for Permissioned Blockchains

Rangsimann Sattayarom, Ratchanon Wongwitutai, Natthawat Tungsriworakan,

Tagrid Chongkolrattanapond, Somchart Fugkeaw

Sirindhorn International Institute of Technology, Thammasat University, Thailand

6622771747@g.siit.tu.ac.th, 6622780268@g.siit.tu.ac.th, 6622772364@g.siit.tu.ac.th, 6622781175@g.siit.tu.ac.th,  
somchart@siit.tu.ac.th

**Abstract**—Redactable permissioned blockchains enable legitimate modification of committed transactions, but controlled mutability introduces two important challenges: efficiently verifying that a requester is authorized to perform a redaction and preserving auditable evidence of modifications after the original ledger state has changed. Existing approaches may require disclosure of authorization information, repeated proof verification, or expensive examination of redaction history. This paper proposes ZK-Redact, an efficient privacy-preserving framework for authorized redaction and provenance auditing in permissioned blockchains. ZK-Redact uses zero-knowledge proofs to allow a requester to demonstrate compliance with a registered redaction policy without revealing the private authorization attributes used to satisfy the policy. To reduce processing overhead, authenticated requests are pre-filtered and their independent proofs are processed through a batch-parallel verification workflow before redaction execution. For each successful redaction, ZK-Redact generates an authenticated provenance record binding the target transaction, previous and updated states, applicable policy, and verified authorization evidence. The resulting provenance records are maintained in an authenticated history structure that enables auditors to efficiently verify current transaction integrity and the sequence of authorized redactions using compact cryptographic proofs. ZK-Redact therefore provides privacy-preserving redaction authorization, scalable proof processing, and efficient integrity and provenance auditing within a unified permissioned-blockchain framework.

**Index Terms**—Redactable Blockchain, Zero-Knowledge Proof, Verifiable Redaction, Redaction Provenance, Integrity Auditing, Permissioned Blockchain.

## I. INTRODUCTION

Permissioned blockchains provide shared and tamper-evident records for multi-organization applications. However, strict immutability can conflict with practical requirements such as correcting erroneous data, removing sensitive information, or satisfying privacy regulations. Redactable blockchains address this limitation by enabling authorized modification while preserving ledger consistency, commonly using chameleon hashes (CHs) [1], [2], [9]. Recent studies have extended CH-based redaction with decentralized authorization, fine-grained access control, anonymity, dynamic updates, and verifiability [4], [6], [14]–[16], [20].

Despite these advances, supporting *large-scale* privacy-preserving redaction remains challenging. First, *privacy-preserving authorization can become a verification bottleneck*. Anonymous and privacy-preserving redaction mechanisms protect requester identity or authorization information [4], [14], [17], but independently processing cryptographic

authorization evidence for many concurrent requests increases verification overhead. Existing scalability-oriented schemes mainly optimize blockchain structures or redaction processing [10], [14], [21], leaving scalable privacy-preserving authorization under massive concurrent requests insufficiently addressed.

Second, *redaction execution can become computationally expensive at scale*. CH-based schemes require authorized collision computation and ledger-state updates for each modification [1], [2], [6], [19], while policy enforcement, distributed authorization, and accountability introduce additional processing [3], [15], [20]. In permissioned implementations, individually processing each validated request may further repeat chaincode, transaction, and ledger-update operations. Consequently, large redaction workloads require an efficient mechanism for organizing validated requests and amortizing common blockchain-processing overhead.

Third, *efficient verification of redaction history remains challenging*. Since a valid CH redaction preserves blockchain linkage, the current ledger state alone does not reveal how a transaction has evolved. Existing schemes support traceability, integrity auditing, and authenticated redaction structures [10], [13], [16], [20]; however, verifying multiple successive redactions may still require locating and examining their corresponding historical records. Efficient auditing therefore requires authenticated provenance that supports direct retrieval and compact verification of both individual redactions and the continuity of a transaction’s redaction history.

To address these challenges, we propose **ZK-Redact**, a privacy-preserving framework for scalable redaction and provenance auditing in permissioned blockchains. ZK-Redact uses state- and policy-bound zero-knowledge proofs (ZKPs) for redaction authorization without revealing private credentials or attributes. Logical proof shards distribute ZKP verification across parallel workers, with intra-shard batching when supported. Verified requests are then batch-organized for CH-based redaction, reducing common chaincode and ledger-processing overhead. Finally, transaction-specific hash-linked provenance is Merkle-authenticated and blockchain-anchored, enabling efficient retrieval and independent verification of redaction history.

The main contributions are summarized as follows:

- **Sharded Privacy-Preserving ZK Authorization:** We introduce state- and policy-bound ZK authorization with logical proof sharding for parallel verification. Intra-shard

batching further amortizes verification when supported, reducing ZKP bottlenecks while preserving request-level authorization and failure isolation.

- **Optimized Batch Redaction Processing:** We separate ZKP verification from redaction execution and batch successfully verified requests for processing. State revalidation prevents stale redactions, while common chaincode, validation, and ledger operations are amortized without aggregating request-specific CH computations.
- **Authenticated Provenance and Verifiable Auditing:** We combine transaction-specific hash-linked histories, Merkle-authenticated entries, and blockchain-anchored commitments to support direct provenance retrieval and independent verification of ZKP authorization, history completeness, ordering, and state continuity without exhaustive blockchain traversal.

## II. RELATED WORK

Existing redactable blockchain research has primarily addressed controlled modification, decentralized authorization, privacy, and verifiability. Ateniese *et al.* [9] established the foundation for blockchain rewriting using chameleon hashes (CHs), while Huang *et al.* [2] extended controlled redaction to consortium blockchains for Industrial Internet of Things (IIoT) environments. Subsequent schemes have reduced dependence on a single trapdoor holder through decentralized or distributed CH constructions [1], [6], [12], [19]. Fine-grained redaction has also been investigated through attribute- and policy-based mechanisms [8], [15], [18], while dynamic redaction and trust-aware authorization have been considered in [5], [16]. These schemes strengthen control over *who* can modify blockchain data, but their primary focus is secure redaction rather than optimizing the complete authorization and redaction pipeline under massive concurrent requests.

Privacy and accountability introduce additional requirements. Huang *et al.* [14] proposed scalable redaction with anonymity, while PriChain [4] supports privacy-preserving fine-grained redaction in decentralized settings. More recently, Wang *et al.* [17] considered anonymous and accountable data redaction for IIoT. These approaches demonstrate the importance of protecting requester information while retaining redaction accountability. However, privacy-preserving authorization introduces additional cryptographic processing, and the cited works do not directly address the use of *proof sharding with batch ZKP verification* for distributing large concurrent authorization workloads. ZK-Redact focuses on this scalability dimension by partitioning independently authenticated proofs across logical shards and processing them in parallel and in batches.

Scalability has also been studied from the redaction and data-structure perspectives. Huang *et al.* [14] considered scalable redactable blockchains with update support, while Wu *et al.* [10] introduced an extended Merkle tree for inserted-data redaction in permissioned blockchains. Fugkeaw *et al.* [21] further investigated efficient redaction for large-scale GDPR-compliant blockchain-based e-KYC. Other schemes support dynamic redaction [5], [19], decentralized CH operations

[1], [6], and multi-party CH constructions [12]. Nevertheless, individually executing authorization, CH-related operations, and blockchain updates can still incur repeated processing as the number of redaction requests grows. In contrast, ZK-Redact separates ZKP authorization from redaction execution and organizes already-validated requests for batch-oriented processing, thereby reducing repeated chaincode and ledger-processing operations.

Verifiability and traceability constitute another important research direction. VRBC [13] provides verifiable redaction with efficient query and integrity auditing, while Zhang *et al.* [16] support update and traceability through dynamic trust-based redaction. Miao *et al.* [20] further combine verifiable redaction with lightweight storage and permission supervision. ReAcct [11] considers redaction control in interoperable blockchain environments, and authenticated tree structures such as EMT [10] improve verification of modified blockchain data. These mechanisms provide important foundations for redaction accountability and integrity verification. However, efficiently retrieving and verifying the *successive provenance history of a particular transaction* remains a distinct requirement: an auditor should be able to determine the ordered sequence of redactions and verify continuity between previous and resulting states without exhaustively traversing blockchain records.

In summary, existing work provides strong foundations for controlled CH-based redaction, privacy-preserving authorization, scalable redaction structures, and integrity auditing. ZK-Redact complements these directions by jointly addressing three processing stages that become critical under large-scale workloads: (i) sharded batch ZKP verification for scalable privacy-preserving authorization, (ii) batch-oriented processing of independently validated redactions, and (iii) authenticated provenance organization for efficient retrieval and verification of successive redaction history.

## III. THE PROPOSED ZK-REDACT FRAMEWORK

This section presents the system model, threat model, and construction of ZK-Redact. The framework is designed to efficiently process privacy-preserving redaction requests while preserving verifiable provenance for subsequent auditing.

### A. System Model

ZK-Redact operates over a permissioned blockchain shared by multiple organizations. As shown in Fig. 1, it consists of six components: *Requester (RQ)*, *Redaction Gateway (RG)*, *Proof Verification Layer (PVL)*, *Permissioned Blockchain (PB)*, *Provenance Audit Index (PAI)*, and *Auditor (AU)*.

- 1) **Requester (RQ):** A registered user or organization requesting modification of a blockchain transaction. The RQ constructs the redaction request and generates a zero-knowledge proof (ZKP) demonstrating compliance with the applicable redaction policy without revealing private authorization information.
- 2) **Redaction Gateway (RG):** The RG serves as the admission-control layer. It authenticates and pre-validates incoming requests, filters stale or duplicate requests,

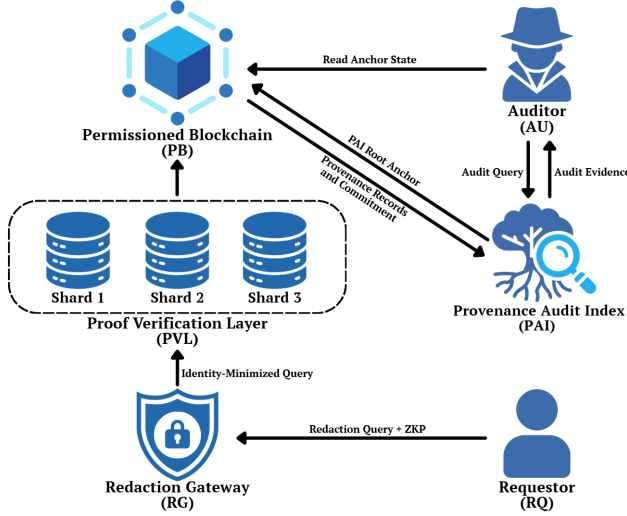


Fig. 1: System model of the proposed ZK-Redact framework

and removes unnecessary requester-identifying information before forwarding accepted requests to the proof-verification layer.

- 3) **Proof Verification Layer (PVL):** The PVL provides scalable ZKP authorization verification. It distributes accepted proofs across logical proof shards for parallel processing and supports intra-shard batched verification when available. Only successfully verified requests proceed to redaction.
- 4) **Permissioned Blockchain (PB):** The PB maintains transactions, redaction policies, and authenticated redaction states. It executes verified redactions using CH-based modification and organizes validated requests for batch-oriented blockchain processing to reduce common chain-code and ledger overhead.
- 5) **Provenance Audit Index (PAI):** The PAI maintains authenticated, transaction-specific redaction histories. It cryptographically links successive redaction states and uses a Merkle-based commitment anchored on the PB to support efficient provenance retrieval and integrity verification.
- 6) **Auditor (AU):** The AU retrieves and independently verifies the provenance of a target transaction. It validates redaction authorization, history integrity and completeness, and successive-state continuity without accessing private ZKP witnesses or traversing unrelated blockchain records.

### B. Threat Model

#### C. System Process

ZK-Redact operates through sequential phases covering system setup, privacy-preserving authorization, sharded ZKP verification, batch redaction, provenance maintenance, and audit verification.

**Phase 1: System Setup:** This phase initializes the cryptographic, authorization, proof-processing, blockchain, and provenance components of ZK-Redact.

TABLE I: Notation

| Symbol                                                               | Description                                                                     |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <i>System Parameters</i>                                             |                                                                                 |
| $N$                                                                  | Number of logical proof shards                                                  |
| $B, \Delta$                                                          | Proof-verification batch size and waiting bound                                 |
| $B_R, \Delta_R$                                                      | Redaction batch size and waiting bound                                          |
| $vk_{zk}, tk_{ch}$                                                   | ZKP verification key, CH trapdoor                                               |
| $S_j, \mathbb{S}$                                                    | Proof shard $j$ and the set of shards                                           |
| <i>Indices and Sets</i>                                              |                                                                                 |
| $i$                                                                  | Transaction index; $TID_i, v_i$ are its identifier and version                  |
| $\rho$                                                               | Redaction-request index                                                         |
| $tx(\rho)$                                                           | Transaction targeted by $\rho$ (many-to-one); written $i$ for fixed $\rho$      |
| $u, a$                                                               | Requester and auditor indices                                                   |
| $q$                                                                  | Policy index; $q_\rho$ is selected by $PID_\rho$ , written $q$ for fixed $\rho$ |
| $j$                                                                  | Proof-shard index                                                               |
| $s$                                                                  | ZKP verification round index                                                    |
| $e$                                                                  | Redaction/anchor round index; $e_k$ is the round recorded in $PR_i^{(k)}$       |
| $k$                                                                  | Historical provenance-record index                                              |
| $v_i^{(e)}$                                                          | Version of transaction $i$ after round $e$                                      |
| $\tilde{v}_i$                                                        | Version returned by the PAI for the audited round                               |
| $\mathcal{I}, \mathcal{I}^{(e)}$                                     | Set of indexed transactions                                                     |
| $\mathcal{T}^{(e)}$                                                  | Transactions redacted in round $e$                                              |
| $\mathcal{Q}_j^{(s)}$                                                | Set of requests verified by shard $j$ in round $s$                              |
| $\mathcal{Q}_R^{(e)}, \mathcal{Q}_F^{(e)}, \mathcal{Q}_{succ}^{(e)}$ | Batched, fresh, and successful request sets of round $e$                        |
| $\mathcal{P}_{ver}$                                                  | Pool of successfully verified requests                                          |
| $\mathcal{B}_j^{(s)}, \mathcal{B}_R^{(e)}$                           | Proof-verification and redaction batches                                        |
| $\tau_j^{(s)}, \tau_R^{(e)}$                                         | Waiting time of the oldest request in a batch                                   |
| <i>Policy and Authorization Evidence</i>                             |                                                                                 |
| $P_q, C_{P_q}$                                                       | Redaction policy and its commitment                                             |
| $\Pi_a^{audit}$                                                      | Registered audit policy of auditor $AU_a$                                       |
| $x_\rho, \pi_\rho$                                                   | ZK public statement and proof                                                   |
| $\eta_\rho$                                                          | Statement-binding digest $H(x_\rho)$                                            |
| $Q_\rho, Q_\rho^*$                                                   | Redaction request and its identity-minimized form                               |
| $DID_\rho, \mathcal{D}$                                              | Deduplication identifier and the set of processed ones                          |
| $C_\rho^{auth}$                                                      | Authorization-evidence commitment                                               |
| $attr_\rho$                                                          | Requester authorization attributes                                              |
| $n_\rho, n_a$                                                        | Requester and auditor nonces                                                    |
| $V_\rho$                                                             | Per-request ZKP verification result                                             |
| <i>Provenance State</i>                                              |                                                                                 |
| $T_i^{(v_i)}$                                                        | Transaction state at version $v_i$                                              |
| $d_i^{(0)}$                                                          | Initial-state digest $H(T_i^{(0)})$                                             |
| $d_i^{old, (k)}, d_i^{new, (k)}$                                     | Pre- and post-state digests recorded in $PR_i^{(k)}$                            |
| $PR_i^{(k)}$                                                         | Provenance record $k$ of transaction $i$                                        |
| $\mathcal{H}_i^{(v_i)}, c_i^{(v_i)}$                                 | Provenance history and cumulative commitment                                    |
| $A_i^{(v_i)}, \mu_i^{(e)}$                                           | PAI entry and its Merkle authentication path                                    |
| $R_{PAI}^{(e)}$                                                      | PAI root after redaction/anchor round $e$                                       |
| $\psi_{i,k}$                                                         | Merkle authentication path for provenance record $k$                            |
| $R_B^{(e)}, R_{PR}^{(e)}, C_B^{(e)}$                                 | Round-binding root, provenance root, and batch-result commitment                |
| $AC^{(e)}, AR^{(e)}$                                                 | Anchor commitment and blockchain anchor record of round $e$                     |

### Step 1: System and Entity Initialization

The system administrator initializes

$$PP = (H, \text{Sig}, \text{CH}, \text{ZK}, N, B, \Delta, B_R, \Delta_R) \quad (1)$$

where  $H$  is a collision-resistant hash function; Sig, CH, and ZK denote the signature, chameleon-hash, and zero-knowledge proof schemes;  $N$  is the number of proof shards; and  $(B, \Delta)$  and  $(B_R, \Delta_R)$  specify the size and waiting bounds for proof-verification and redaction batches, respectively.

Each requester  $U_u$  generates

$$(pk_u, sk_u) \leftarrow \text{Sig.KeyGen}(1^\lambda) \quad (2)$$

and registers  $E_u = (ID_u, pk_u)$ , while retaining  $sk_u$  for request authentication.

Similarly, each auditor  $AU_a$  generates

$$(pk_a, sk_a) \leftarrow \text{Sig.KeyGen}(1^\lambda) \quad (3)$$

and registers

$$E_a^{\text{AU}} = (AID_a, pk_a, \Pi_a^{\text{audit}}) \quad (4)$$

where  $\Pi_a^{\text{audit}}$  specifies its permitted audit scope. These registered credentials are subsequently used to authenticate and authorize provenance queries in Phase 6.

### Step 2: Redaction Policy Registration

For each protected transaction class, the policy administrator defines

$$P_q = (PID_q, \mathcal{A}_q, \mathcal{C}_q, \mathcal{W}_q, \mathcal{V}_q) \quad (5)$$

where  $\mathcal{A}_q, \mathcal{C}_q, \mathcal{W}_q$ , and  $\mathcal{V}_q$  specify the authorization predicates, additional conditions, permitted redaction scope, and validity period. Its commitment

$$C_{P_q} = H(PID_q \parallel \mathcal{A}_q \parallel \mathcal{C}_q \parallel \mathcal{W}_q \parallel \mathcal{V}_q) \quad (6)$$

is recorded on the blockchain and subsequently bound to the public ZKP statement.

### Step 3: ZKP and Chameleon-Hash Setup

The system generates

$$(pp_{\text{zk}}, vk_{\text{zk}}) \leftarrow \text{ZK.Setup}(1^\lambda, \mathcal{R}_{\text{red}}) \quad (7)$$

where  $\mathcal{R}_{\text{red}}$  defines policy-compliant redaction authorization. The proving parameters  $pp_{\text{zk}}$  are available to requesters, while  $vk_{\text{zk}}$  is provided to the PVL.

The CH parameters are generated as

$$(pk_{\text{ch}}, tk_{\text{ch}}) \leftarrow \text{CH.KeyGen}(1^\lambda) \quad (8)$$

where  $tk_{\text{ch}}$  is protected by the trusted redaction component and used only for authorized CH adaptation.

### Step 4: Blockchain and Proof-Shard Initialization

The permissioned blockchain deploys chaincode for entity and policy registration, audit-access authorization, transaction updates, redaction results, and provenance commitments. The PVL initializes

$$\mathbb{S} = \{\mathcal{S}_1, \mathcal{S}_2, \dots, \mathcal{S}_N\} \quad (9)$$

where each  $\mathcal{S}_j$  maintains an independent proof queue. Runtime shard assignment and verification-batch formation are performed in Phase 3.

### Step 5: Provenance Audit Index Initialization

For each transaction  $TID_i$  with authenticated initial state  $T_i^{(0)}$ , the PAI initializes

$$v_i = 0, \quad \mathcal{H}_i^{(0)} = \emptyset \quad (10)$$

and computes

$$d_i^{(0)} = H(T_i^{(0)}) \quad (11)$$

$$c_i^{(0)} = H(TID_i \parallel 0 \parallel d_i^{(0)}) \quad (12)$$

The corresponding authenticated entry is

$$A_i^{(0)} = H(TID_i \parallel 0 \parallel c_i^{(0)}) \quad (13)$$

and the initial PAI root is

$$R_{\text{PAI}}^{(0)} = \text{MerkleRoot}\left(\left\{A_i^{(0)}\right\}_{i \in \mathcal{I}}\right) \quad (14)$$

where  $\mathcal{I}$  is the set of indexed transactions.  $R_{\text{PAI}}^{(0)}$  is anchored to the permissioned blockchain as the trusted initial provenance state.

**Phase 2: Redaction Request and Privacy-Preserving Authorization:** This phase authenticates each redaction request, proves policy compliance without revealing private authorization attributes, and filters invalid, stale, or duplicate requests before ZKP verification.

### Step 1: Redaction Request and Authentication

For target transaction  $i = \text{tx}(\rho)$  with identifier  $TID_i$  at version  $v_i$ , requester  $U_u$  constructs

$$R_\rho = (TID_i, v_i, Loc_\rho, M_\rho, PID_\rho, t_\rho, n_\rho) \quad (15)$$

where  $Loc_\rho$  and  $M_\rho$  specify the target location and modification,  $PID_\rho$  identifies the applicable policy  $P_q$ , and  $(t_\rho, n_\rho)$  provides freshness. The requester authenticates the request by

$$\sigma_\rho = \text{Sig.Sign}(sk_u, H(R_\rho)) \quad (16)$$

binding it to the target state, operation, policy, and freshness information.

### Step 2: Policy- and State-Bound ZK Authorization

The requester obtains the current policy version  $v_{P_q}$  and commitment  $C_{P_q}$ , and constructs the public ZKP statement

$$x_\rho = (TID_i, v_i, Loc_\rho, M_\rho, PID_\rho, v_{P_q}, C_{P_q}, t_\rho, n_\rho) \quad (17)$$

with compact binding digest

$$\eta_\rho = H(x_\rho) \quad (18)$$

The private witness is

$$w_\rho = (Cred_u, \text{attr}_\rho, \zeta_\rho) \quad (19)$$

where  $Cred_u$  is the requester's credential,  $\text{attr}_\rho$  its private authorization attributes, and  $\zeta_\rho$  auxiliary credential evidence.

The authorization relation

$$\mathcal{R}_{\text{red}}(x_\rho, w_\rho) = 1 \quad (20)$$

holds iff  $Cred_u$  is valid and non-revoked,  $\text{attr}_\rho$  satisfies the predicates represented by  $C_{P_q}$ ,  $(Loc_\rho, M_\rho)$  is within the permitted scope, and the transaction and policy versions and freshness values are valid. Thus, the proof is bound to the specific redaction request and its governing policy.

The requester generates

$$\pi_\rho \leftarrow \text{ZK.Prove}(pp_{\text{zk}}, x_\rho, w_\rho) \quad (21)$$

and submits

$$Q_\rho = (R_\rho, v_{P_q}, C_{P_q}, \sigma_\rho, \pi_\rho) \quad (22)$$

to the Redaction Gateway (RG).

### Step 3: Gateway Pre-Validation and Deduplication

The RG retrieves the registered  $pk_u$  of the issuing requester and verifies

$$\text{Sig.Verify}(pk_u, H(R_\rho), \sigma_\rho) = 1 \quad (23)$$

It then validates  $(t_\rho, n_\rho)$ , confirms that  $v_i$  is the current version of  $\text{tx}(\rho)$ , and checks the registered policy tuple  $(PID_\rho, v_{P_q}, C_{P_q})$ . Invalid or stale requests are rejected before ZKP verification.

For each accepted request, the RG computes

$$DID_\rho = H(TID_i \parallel v_i \parallel Loc_\rho \parallel M_\rho \parallel PID_\rho) \quad (24)$$

and accepts it only if

$$DID_\rho \notin \mathcal{D} \quad (25)$$

where  $\mathcal{D}$  contains pending or previously processed identifiers. The accepted  $DID_\rho$  is then inserted into  $\mathcal{D}$ . Including  $v_i$  prevents duplicate processing against the same transaction state while permitting a new redaction after the state advances.

### Step 4: Identity-Minimized Downstream Submission

After pre-validation, the RG removes the explicit requester identity and constructs

$$Q_\rho^* = (DID_\rho, x_\rho, \eta_\rho, \pi_\rho) \quad (26)$$

Since  $x_\rho$  already contains the transaction, operation, policy, version, and freshness information required for downstream processing, this compact representation avoids duplicating those fields.

The RG securely retains the mapping between  $DID_\rho$  and the authenticated requester  $U_u$  for accountability, while the PVL and downstream components receive only  $Q_\rho^*$ . Thus, explicit requester identity is hidden from downstream processing without assuming anonymity against the RG.

The accepted  $Q_\rho^*$  is forwarded to Phase 3 for deterministic proof-shard assignment and parallel ZKP verification.

**Phase 3: Sharded Batch ZKP Verification:** This phase distributes accepted ZK authorization proofs across logical proof shards for parallel verification, combining inter-shard parallelism with intra-shard batched processing.

#### Step 1: Proof-Shard Assignment

For each accepted query  $Q_\rho^*$  from Phase 2, the PVL assigns

$$j_\rho = 1 + (\text{Int}(H(DID_\rho)) \bmod N), \quad Q_\rho^* \rightarrow \mathcal{S}_{j_\rho} \quad (27)$$

where  $N$  is the number of logical proof shards. The hash-based rule provides uniform workload distribution in expectation. Each shard  $\mathcal{S}_j$  maintains an independent verification queue and is processed concurrently with other active shards.

#### Step 2: Shard-Level Batch Formation

Each active shard forms

$$\mathcal{B}_j^{(s)} = \{(x_\rho, \pi_\rho, Q_\rho^*)\}_{\rho \in \mathcal{Q}_j^{(s)}}, \quad |\mathcal{B}_j^{(s)}| \leq B \quad (28)$$

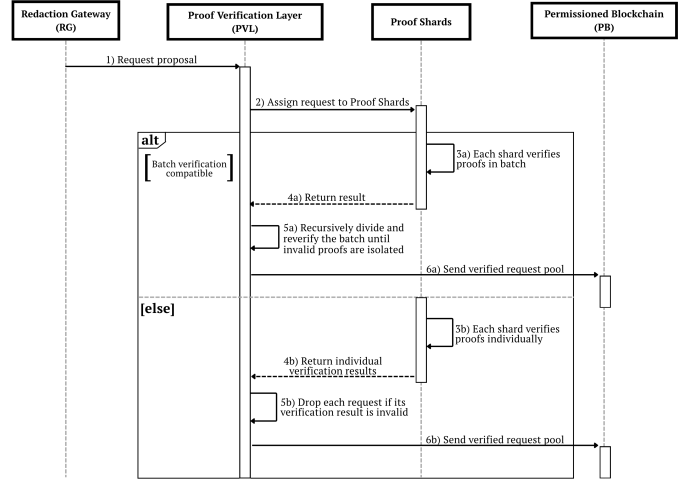


Fig. 2: Sequence of redaction-request verification using distributed proof shards

where  $\mathcal{Q}_j^{(s)}$  is the set of requests selected in verification round  $s$ . A batch closes when

$$\text{Close}(\mathcal{B}_j^{(s)}) = (|\mathcal{B}_j^{(s)}| = B) \vee (\tau_j^{(s)} \geq \Delta) \quad (29)$$

where  $\tau_j^{(s)}$  is the waiting time of the oldest request. This permits larger batches under high load while bounding delay under sparse arrivals.

### Step 3: Parallel ZKP Verification

Let  $\mathcal{J}_s$  denote the active shards in round  $s$ . Their batches are processed concurrently, while each request is verified as

$$V_\rho = \text{ZK.Verify}(vk_{zk}, x_\rho, \pi_\rho), \quad V_\rho \in \{0, 1\} \quad (30)$$

Thus, proof sharding provides parallelism independently of any scheme-specific batch-verification capability.

If the instantiated ZKP scheme supports native batch verification, shard  $\mathcal{S}_j$  may instead compute

$$\mathbf{V}_j^{(s)} \leftarrow \text{ZK.BatchVerify}(vk_{zk}, \{(x_\rho, \pi_\rho)\}_{\rho \in \mathcal{Q}_j^{(s)}}) \quad (31)$$

where

$$\mathbf{V}_j^{(s)} = \{V_\rho\}_{\rho \in \mathcal{Q}_j^{(s)}} \quad (32)$$

Otherwise, the proofs in  $\mathcal{B}_j^{(s)}$  are independently verified using (30). Hence, native batch verification is an optional optimization rather than a correctness requirement.

### Step 4: Failure Isolation and Verified Request Pool

Verification remains request-specific, and  $Q_\rho^*$  is accepted only if  $V_\rho = 1$ . If a native batch verifier returns only an aggregate pass/fail result, a failed batch is recursively divided and reverified until invalid proofs are isolated. This step is unnecessary when per-proof results are available.

Successfully verified requests form

$$\mathcal{P}_{\text{ver}} = \{Q_\rho^* \mid V_\rho = 1\} \quad (33)$$

Invalid requests are rejected and their pending identifiers are released according to the request-management policy.

Phase 3 therefore uses *proof sharding* for parallel ZKP verification and *intra-shard batching* for efficient scheduling and, when supported, native batch-verification gains.  $\mathcal{P}_{\text{ver}}$  is forwarded to Phase 4 for redaction execution. Verification rounds  $s$  and redaction/anchor rounds  $e$  are independent logical clocks. Successfully verified requests are decoupled from both clocks through  $\mathcal{P}_{\text{ver}}$ .

**Phase 4: Batch-Verified Redaction and Provenance Generation:** This phase organizes verified requests for batch-oriented processing, executes authorized CH redactions, and generates authenticated provenance evidence. CH adaptation remains request-specific, while common blockchain operations are amortized across the batch.

#### Step 1: Redaction Batch Formation

Verified requests in  $\mathcal{P}_{\text{ver}}$  are organized as

$$\mathcal{B}_R^{(e)} = \{Q_\rho^*\}_{\rho \in \mathcal{Q}_R^{(e)}}, \quad |\mathcal{B}_R^{(e)}| \leq B_R \quad (34)$$

where  $B_R$  is the maximum batch size and  $\mathcal{Q}_R^{(e)}$  is the set of requests drawn from  $\mathcal{P}_{\text{ver}}$  in round  $e$ . A batch closes when

$$\text{Close}(\mathcal{B}_R^{(e)}) = (|\mathcal{B}_R^{(e)}| = B_R) \vee (\tau_R^{(e)} \geq \Delta_R) \quad (35)$$

Requests targeting different transactions execute independently. Requests sharing a target transaction are ordered; since  $x_\rho$  binds the target version  $v_i$ , only the first can pass revalidation, and the remainder become stale and must be reauthorized. At most one redaction per transaction therefore succeeds in a round.

#### Step 2: State Revalidation and Execution Commitment

Immediately before execution, the PB checks for each  $\rho \in \mathcal{Q}_R^{(e)}$

$$\text{Fresh}_\rho = [v_i = v_i^{\text{cur}}] \wedge [v_{P_q} = v_{P_q}^{\text{cur}}] \quad (36)$$

where  $v_i^{\text{cur}}$  and  $v_{P_q}^{\text{cur}}$  are the current transaction and policy versions. A stale request is excluded and must be reauthorized.

The fresh execution set is

$$\mathcal{Q}_F^{(e)} = \{\rho \in \mathcal{Q}_R^{(e)} : \text{Fresh}_\rho = 1\} \quad (37)$$

and is bound to the processing round by

$$R_B^{(e)} = \text{MerkleRoot} \left( \{H(DID_\rho \parallel \eta_\rho)\}_{\rho \in \mathcal{Q}_F^{(e)}} \right) \quad (38)$$

where  $\eta_\rho = H(x_\rho)$  is the statement-binding digest defined in Phase 2.

#### Step 3: Authorized CH Redaction

For each  $\rho \in \mathcal{Q}_F^{(e)}$ , let  $m_i^{(v_i)}$  and  $r_i^{(v_i)}$  denote the current content and CH randomness of transaction  $i$ . For modified content  $m_i^{(v_i+1)}$ , the trusted redaction component computes

$$r_i^{(v_i+1)} \leftarrow \text{CH.Adapt} \left( tk_{\text{ch}}, m_i^{(v_i)}, r_i^{(v_i)}, m_i^{(v_i+1)} \right) \quad (39)$$

such that

$$\text{CH.Hash}(pk_{\text{ch}}, m_i^{(v_i)}, r_i^{(v_i)}) = \text{CH.Hash}(pk_{\text{ch}}, m_i^{(v_i+1)}, r_i^{(v_i+1)}) \quad (40)$$

The authorized state transition is

$$T_i^{(v_i)} \longrightarrow T_i^{(v_i+1)} \quad (41)$$

Thus the value committed for a redactable transaction in the block Merkle tree is its chameleon digest  $\text{CH.Hash}(pk_{\text{ch}}, m_i^{(v_i)}, r_i^{(v_i)})$  rather than  $H(m_i^{(v_i)})$ , the collision in (40) leaves the committed leaf, and therefore  $h_{\text{merkle}}$ , the block hash, and all forward links, unchanged. Thus, CH adaptation is performed independently per request, while batching amortizes common chaincode, validation, commitment, and ledger processing.

#### Step 4: Provenance and Audit-Evidence Generation

For each successful redaction by request  $\rho$ , ZK-Redact computes

$$d_i^{\text{old},(v_i+1)} = H(T_i^{(v_i)}) \quad (42)$$

$$d_i^{\text{new},(v_i+1)} = H(T_i^{(v_i+1)}) \quad (43)$$

and binds its authorization evidence by

$$C_\rho^{\text{auth}} = H(DID_\rho \parallel \eta_\rho \parallel H(\pi_\rho)) \quad (44)$$

It constructs

$$PR_i^{(v_i+1)} = (TID_i, v_i, v_i + 1, PID_\rho, v_{P_q}, DID_\rho, e, d_i^{\text{old},(v_i+1)}, d_i^{\text{new},(v_i+1)}, C_\rho^{\text{auth}}, R_B^{(e)}, t'_\rho) \quad (45)$$

where  $t'_\rho$  is the completion timestamp. The public audit evidence  $(x_\rho, \pi_\rho)$  is retained off-chain under  $DID_\rho$  and authenticated by  $C_\rho^{\text{auth}}$ , enabling independent verification in Phase 6.

#### Step 5: Batch-Result Commitment

Let

$$\mathcal{Q}_{\text{succ}}^{(e)} \subseteq \mathcal{Q}_F^{(e)} \quad (46)$$

denote the successfully executed requests and

$$\mathcal{T}^{(e)} = \{\text{tx}(\rho) : \rho \in \mathcal{Q}_{\text{succ}}^{(e)}\} \subseteq \mathcal{I}^{(e)} \quad (47)$$

the transactions they redact. Since at most one request per transaction succeeds in a round,  $|\mathcal{T}^{(e)}| = |\mathcal{Q}_{\text{succ}}^{(e)}|$ . ZK-Redact computes

$$R_{PR}^{(e)} = \text{MerkleRoot} \left( \left\{ H(PR_i^{(v_i+1)}) \right\}_{i \in \mathcal{T}^{(e)}} \right) \quad (48)$$

$$C_B^{(e)} = H \left( R_B^{(e)} \parallel e \parallel R_{PR}^{(e)} \right) \quad (49)$$

The PB commits the successful state updates and  $C_B^{(e)}$ ; stale or failed requests are excluded without affecting independent successful redactions. The resulting provenance records are forwarded to the PAI for authenticated maintenance in Phase 5.

Phase 4 therefore separates request-specific CH adaptation from batch-level blockchain processing, preserving per-request correctness while reducing common processing overhead.

**Phase 5: Authenticated Provenance Maintenance:** This phase updates the transaction-specific provenance histories after each completed redaction batch and anchors the resulting PAI state to the blockchain. Processing round  $e$  corresponds to the successful batch-result commitment  $C_B^{(e)}$  generated in Phase 4.

#### Step 1: Provenance-Chain Update

For each successful redaction  $PR_i^{(v_i+1)}$  of round  $e$ , with  $i \in \mathcal{T}^{(e)}$ , the PAI appends

$$\mathcal{H}_i^{(v_i+1)} = \mathcal{H}_i^{(v_i)} \parallel PR_i^{(v_i+1)} \quad (50)$$

and updates its cumulative commitment as

$$c_i^{(v_i+1)} = H\left(c_i^{(v_i)} \parallel H(PR_i^{(v_i+1)})\right) \quad (51)$$

where  $c_i^{(0)}$  is initialized in Phase 1. The resulting version is

$$v_i^{(e)} = \begin{cases} v_i + 1, & i \in \mathcal{T}^{(e)} \\ v_i, & \text{otherwise} \end{cases} \quad (52)$$

Successive records must satisfy

$$d_i^{\text{new},(k)} = d_i^{\text{old},(k+1)}, \quad 1 \leq k < v_i^{(e)} \quad (53)$$

together with the lower-boundary condition

$$d_i^{\text{old},(1)} = H(T_i^{(0)}) \quad (54)$$

thereby binding each redaction output to the next input state and the start of the history to the authenticated initial state.

### Step 2: Transaction-Specific Index Update

After all records of round  $e$  for transaction  $i \in \mathcal{T}^{(e)}$  have been applied, the PAI computes a single updated entry

$$A_i^{(v_i^{(e)})} = H\left(TID_i \parallel v_i^{(e)} \parallel c_i^{(v_i^{(e)})}\right) \quad (55)$$

The key  $TID_i$  provides direct access to  $\mathcal{H}_i$ , while the version and cumulative commitment bind its expected length, ordering, and terminal history state. Transactions  $i \notin \mathcal{T}^{(e)}$  retain their round- $(e-1)$  entries unchanged.

### Step 3: PAI Authentication

After all successful records of round  $e$  are incorporated, the PAI computes

$$R_{\text{PAI}}^{(e)} = \text{MerkleRoot}\left(\left\{A_i^{(v_i^{(e)})}\right\}_{i \in \mathcal{I}^{(e)}}\right) \quad (56)$$

where  $\mathcal{I}^{(e)}$  is the set of indexed transactions. The tree therefore contains exactly one leaf per indexed transaction, updated for  $i \in \mathcal{T}^{(e)}$  and carried forward otherwise. A target entry admits the authentication path

$$\mu_i^{(e)} = \text{MerkleProof}\left(A_i^{(v_i^{(e)})}, R_{\text{PAI}}^{(e)}\right) \quad (57)$$

### Step 4: Blockchain Anchoring

After each completed redaction round  $e$ , ZK-Redact computes

$$AC^{(e)} = H\left(e \parallel R_{\text{PAI}}^{(e)} \parallel R_{\text{PAI}}^{(e-1)} \parallel C_B^{(e)}\right) \quad (58)$$

and constructs

$$AR^{(e)} = \left(e, R_{\text{PAI}}^{(e)}, R_{\text{PAI}}^{(e-1)}, C_B^{(e)}, AC^{(e)}\right) \quad (59)$$

The permissioned blockchain records  $AR^{(e)}$  as the trusted provenance anchor. This jointly binds the updated PAI state, its predecessor, and the successful batch-result commitment  $C_B^{(e)}$  to redaction round  $e$ , enabling independent anchor verification in Phase 6.

Phase 5 therefore maintains hash-linked, transaction-specific provenance and authenticates each completed redaction round through a blockchain-anchored PAI root. The authenticated version and terminal commitment enable Phase 6 to detect missing, modified, reordered, or truncated provenance records without scanning unrelated blockchain history.

### Phase 6: Provenance Retrieval and Audit Verification:

This phase enables an authorized auditor to retrieve and independently verify a transaction's redaction history without scanning unrelated blockchain records.

#### Step 1: Auditor Authentication and Query Authorization

Before retrieving provenance, auditor  $AU_a$  constructs

$$R_a^{\text{audit}} = (AID_a, TID_i, e, t_a, n_a) \quad (60)$$

where  $AID_a$  is the registered auditor identifier,  $e$  is the target PAI state, and  $(t_a, n_a)$  provides freshness. The auditor signs

$$\sigma_a = \text{Sig.Sign}(sk_a, H(R_a^{\text{audit}})) \quad (61)$$

Upon receiving  $(R_a^{\text{audit}}, \sigma_a)$ , the permissioned blockchain verifies

$$\text{Sig.Verify}(pk_a, H(R_a^{\text{audit}}), \sigma_a) = 1 \quad (62)$$

using the registered public key  $pk_a$ , and checks the freshness of  $(t_a, n_a)$ . The audit-access chaincode then evaluates

$$\text{AuditAuth}(AID_a, TID_i, e, \Pi_a^{\text{audit}}) = 1 \quad (63)$$

where  $\Pi_a^{\text{audit}}$  is the registered audit policy of  $AU_a$ . The query proceeds only if the signature, freshness, and audit-authorization checks are valid; otherwise, it is rejected before provenance retrieval.

#### Step 2: Provenance and Audit-Evidence Retrieval

Once authenticated, for target  $TID_i$  and PAI state  $e$ , the auditor submits the retrieval query

$$Q_i^{\text{audit}} = (TID_i, e) \quad (64)$$

The PAI returns

$$\mathcal{E}_i^{(e)} = \left(\mathcal{H}_i^{(\bar{v}_i)}, \bar{v}_i, c_i^{(\bar{v}_i)}, A_i^{(\bar{v}_i)}, \mu_i^{(e)}, \{\psi_{i,k}\}_{1 \leq k \leq \bar{v}_i}\right) \quad (65)$$

$$\psi_{i,k} = \text{MerkleProof}\left(H(PR_i^{(k)}), R_{PR}^{(e_k)}\right) \quad (66)$$

Using  $DID_{i,k}$  and  $C_{i,k}^{\text{auth}}$  from  $PR_i^{(k)}$ , the auditor retrieves the public evidence  $(x_{i,k}, \pi_{i,k})$  from the off-chain audit-evidence store and the anchored root  $R_{PR}^{(e_k)}$  from  $AR^{(e_k)}$ .

From the permissioned blockchain, the auditor independently obtains the authenticated initial and round- $e$  transaction states and the anchor record

$$AR^{(e)} = \left(e, R_{\text{PAI}}^{(e)}, R_{\text{PAI}}^{(e-1)}, C_B^{(e)}, AC^{(e)}\right) \quad (67)$$

#### Step 3: PAI and History Verification

The auditor recomputes

$$\hat{A}_i^{(\bar{v}_i)} = H(TID_i \parallel \bar{v}_i \parallel c_i^{(\bar{v}_i)}) \quad (68)$$

and verifies

$$\text{MerkleVerify}\left(\hat{A}_i^{(\bar{v}_i)}, \mu_i^{(e)}, R_{\text{PAI}}^{(e)}\right) = 1 \quad (69)$$



Since  $TID_i$ ,  $\bar{v}_i$ , and  $c_i^{(\bar{v}_i)}$  are all inside the hashed entry, (69) pins the reported version and commitment to those committed in round  $e$ , giving  $\bar{v}_i = v_i^{(e)}$ . The auditor then checks each record against the provenance root of its own round,

$$\text{MerkleVerify}\left(H(PR_i^{(k)}), \psi_{i,k}, R_{PR}^{(e_k)}\right) = 1, \quad 1 \leq k \leq \bar{v}_i \quad (70)$$

Starting from authenticated  $T_i^{(0)}$ , it reconstructs

$$\hat{c}_i^{(0)} = H(TID_i \parallel 0 \parallel H(T_i^{(0)})) \quad (71)$$

and, for  $1 \leq k \leq \bar{v}_i$ ,

$$\hat{c}_i^{(k)} = H(\hat{c}_i^{(k-1)} \parallel H(PR_i^{(k)})) \quad (72)$$

History integrity and completeness require

$$|\mathcal{H}_i^{(\bar{v}_i)}| = \bar{v}_i, \quad \hat{c}_i^{(\bar{v}_i)} = c_i^{(\bar{v}_i)} \quad (73)$$

Thus, missing, inserted, modified, reordered, or truncated records cannot match the authenticated PAI entry. **Step 4:**

#### Authorization and State-Continuity Verification

For each  $PR_i^{(k)}$ , the auditor computes

$$\eta_{i,k} = H(x_{i,k}) \quad (74)$$

$$\hat{C}_{i,k}^{\text{auth}} = H(DID_{i,k} \parallel \eta_{i,k} \parallel H(\pi_{i,k})) \quad (75)$$

and accepts its authorization only if

$$\hat{C}_{i,k}^{\text{auth}} = C_{i,k}^{\text{auth}} \quad \wedge \quad \text{ZK.Verify}(vk_{zk}, x_{i,k}, \pi_{i,k}) = 1 \quad (76)$$

The auditor also checks the policy version recorded in  $PR_i^{(k)}$ .

For successive redactions, the auditor verifies

$$d_i^{\text{new},(k)} = d_i^{\text{old},(k+1)}, \quad 1 \leq k < \bar{v}_i \quad (77)$$

and, for the version pair recorded in each  $PR_i^{(k)}$ ,

$$v_i^{\text{new},(k)} = v_i^{\text{old},(k)} + 1 = v_i^{\text{old},(k+1)}, \quad 1 \leq k < \bar{v}_i \quad (78)$$

It further verifies the boundary states as

$$d_i^{\text{old},(1)} = H(T_i^{(0)}) \quad (79)$$

$$d_i^{\text{new},(\bar{v}_i)} = H(T_i^{(\bar{v}_i)}) \quad (80)$$

where  $T_i^{(0)}$  is the authenticated initial state and  $T_i^{(\bar{v}_i)}$  is the authenticated transaction state at the version certified by the audited PAI state  $e$ . The first condition binds the recorded pre-state of  $PR_i^{(1)}$  to the authenticated initial state, preventing a forged history from starting at an arbitrary digest.

#### Step 5: Anchor Verification

Finally, the auditor verifies

$$AC^{(e)} \stackrel{?}{=} H\left(e \parallel R_{\text{PAI}}^{(e)} \parallel R_{\text{PAI}}^{(e-1)} \parallel C_B^{(e)}\right) \quad (81)$$

This binds the authenticated PAI state to its predecessor and the successful batch-result commitment of redaction round  $e$ .

The audit succeeds only if the PAI membership, provenance commitment, ZKP authorization, state continuity, and blockchain anchor are valid. Thus, ZK-Redact supports transaction-specific verification of redaction authorization, his-

tory integrity, completeness, and continuity without exhaustive blockchain traversal.

## IV. SECURITY ANALYSIS

### V. PERFORMANCE EVALUATION

#### A. Complexity Analysis

We compare ZK-Redact against [10], which redacts inserted data via an extended Merkle tree with committee voting; [13], which provides verifiable redaction through a blockchain authentication tree; and [22], which combines a double-trapdoor chameleon hash with a universal accumulator. Table II summarizes the comparison.

#### B. Experimental Setup

**Implementation:** ZK-Redact and the three baselines are implemented in Go over a common cryptographic substrate, so that no scheme gains an advantage from a library or a security level rather than from its design. All primitives target 128-bit security: SHA-256 for hashing, Groth16 over BLS12-381 for the authorization proof  $\pi_\rho$ , P-256 for the chameleon hash and Schnorr signatures, and a 3072-bit RSA group for the universal accumulator of [22]. The permissioned ledger is Hyperledger Fabric with four organizations, two peers each, and a three-node Raft ordering service. Block cutting is held constant for every system at 100 transactions per block and a 1000ms timeout, since block cadence directly affects redaction commit latency and varying it between systems would invalidate the comparison. All runs execute on one 32-vCPU, 64 GB Ubuntu 22.04 host, with each peer limited to one vCPU and 2 GB so that results do not drift with host load.

**Dataset and Request Trace:** A single seeded generator produces one corpus that every system ingests into its own native representation: 100,000 transactions, each carrying a 256-byte immutable core payload and a 512-byte redactable payload that is the redaction target; 100 registered requesters across the four organizations; and 10 redaction policies of predicate depth three, so that the policy circuit evaluates a realistic conjunction of membership, role, and policy version rather than a single constant. Ingestion is not timed: measurement begins only once every system holds an equivalent ledger derived from identical source data. All systems then replay the *same* trace of 5,000 redaction requests in the same order under a closed-loop arrival model, which makes saturation points unambiguous. The trace controls the concurrency level, the target distribution over transactions, and the *conflict ratio*—the fraction of requests targeting an already-targeted transaction, swept over  $\{0, 0.25, 0.50\}$ —because ZK-Redact serializes same-transaction requests in Phase 4 while independent transactions proceed in parallel.

**Baselines:** ZK-Redact is compared against [10], [13], and [22], each implementing the authorization, redaction, and audit path its paper specifies. Their authorization work differs by design: [10] distributes a request to a voting committee and verifies the aggregated Schnorr votes, whereas [13] and [22] verify possession of a manager trapdoor and of a regulator key. Neither of the latter two defines a per-request authorization



TABLE II: Computation and Auditing Cost Comparison

| Scheme           | Privacy-Preserving<br>Authorization | Authorization<br>Verification Cost                                    | On-Chain Operations<br>per Request | Provenance<br>Audit Cost      | Ledger-Independent<br>Provenance Audit |
|------------------|-------------------------------------|-----------------------------------------------------------------------|------------------------------------|-------------------------------|----------------------------------------|
| [10]             | ×                                   | $c \text{Sig}_v + \text{Consensus}$                                   | $\leq c + 2$                       | $O(Lm + \nu c)$               | No                                     |
| [13]             | ×                                   | —                                                                     | 1                                  | $O(L)$ traversal              | No                                     |
| [22]             | ×                                   | —                                                                     | 1                                  | $O(L)$ traversal              | No                                     |
| <b>ZK-Redact</b> | ✓                                   | $\text{Sig}_v + 3\text{Pair} + \ell_x \text{Exp} (/N \text{ shards})$ | $3/B_R$                            | $O(\nu + \log  \mathcal{I} )$ | <b>Yes</b>                             |

$c$ : committee size in [10];  $L$ : ledger length in blocks;  $m$ : transactions per block;  $\nu$ : number of redactions on the audited transaction;  $\ell_x$ : number of public inputs in  $x_p$ ;  $N$ : proof shards;  $B_R$ : redaction batch size;  $|\mathcal{I}|$ : number of indexed transactions.

protocol, so their lower cost in Experiment 1 is a property of those schemes rather than a handicap imposed here; it is therefore reported together with the capability comparison of Table II, so that the speed can be read against what it is bought with.

**Measurement Boundary:** Cryptographic cost and ledger cost are measured separately and never summed into a single headline number. Cryptographic work is measured off-chain and in-process for all four systems; ledger and consensus cost is reported separately. This follows the methodology of the baselines themselves— [13] reports off-chain computation separately from on-chain gas, [22] excludes communication from its functional measurements, and [10] excludes the redaction consensus—and is established practice for layer-wise benchmarking of permissioned blockchains. Because authorization in [10] is ordered on the ledger while the other three authorize in-process by design, [10] is reported on both transports: an in-process arm as the like-for-like control, and a deployed Fabric arm as its true deployed cost.

**Reproducibility:** Every parameter is read at runtime from one shared configuration file, and each result records the fully resolved configuration, the random seed, the dataset identifier, the code revision, and the environment fingerprint. The same configuration and seed reproduce the same results, and no reported number is obtained from an analytical model or an extrapolation.

### C. Performance Analysis

**Experiment 1: Verification Throughput: Objective.** Determine how many redaction requests per second each system can *authorize*, and how that rate scales with concurrent load. Measurement starts when a request is admitted to the authorization component and stops when the decision is final; execution and ledger commitment belong to Experiment 2.

**Configuration.** Offered concurrency is the primary axis, swept geometrically from 1 to 1024: it starts at 1 because that is the regime in which the trapdoor-based baselines legitimately lead, and extends to 1024 so that the expected crossover falls inside the measured range. ZK-Redact is plotted at shard counts  $N \in \{1, 8, 64\}$ , spanning the disabled, moderate, and oversubscribed settings of the full sweep  $N \in \{1, 2, 4, 8, 16, 32, 64\}$ ; the shard sweep exceeds the available cores so that saturation is demonstrated rather than assumed. The proof-verification batch size  $B \in \{1, 2, 4, 8, 16, 32, 64\}$  with waiting bound  $\Delta \in \{1, 6, 13, 63\}$  ms is swept and reported in Table ???. Since both  $N = 1$  and  $B = 1$  are included, the four-cell ablation grid—neither mechanism,

Fig. 3: Experiment 1: authorization throughput versus number of concurrent requests. Baselines are shown alongside ZK-Redact at shard counts  $N \in \{1, 8, 64\}$ , where  $N = 1$  is the sharding-disabled ablation.

sharding alone, batching alone, and both—falls out of the cross product at every magnitude, which is what makes the Phase 3 claim testable: that sharding supplies parallelism *independently* of native batch-verification support.

**Metrics.** Authorized requests per second is the primary metric, plotted in Fig. 3 on logarithmic axes so that both the low-concurrency and the saturated regimes are legible in one view. Table ?? reports the quantities sharing this sweep: authorization latency at the 50th, 95th, and 99th percentiles; scaling efficiency, the measured speedup divided by  $N$ ; and the saturation point, the offered load beyond which throughput no longer rises. Each point also records the consensus blocks, round trips, and signature verifications its authorization consumed, so that the comparison can be re-derived under a different block interval.

**Results.** [To be completed from the measurement run. Report both regimes: the low-concurrency regime, in which a trapdoor or key check is cheaper than a proof verification and the baselines lead, and the high-concurrency regime, in which their single authorizing entity becomes a serialization point while ZK-Redact distributes verification across shards. State the crossover concurrency against each baseline, the saturation point and scaling efficiency at each shard count, and the separated contributions of sharding and batching from the ablation grid.]

**Experiment 2: Redaction Processing Overhead: Objective.** Quantify the per-request blockchain cost of executing an authorized redaction, how much of it batch-oriented processing amortizes, where the benefit stops, and what it costs in request staleness. Measurement starts when an already-authorized request enters redaction execution and stops when the state transition is committed and the provenance record is generated; authorization is excluded, since counting it twice would overstate the batching benefit.

**Configuration.** The workload axis is the number of redaction requests replayed from the shared trace, grown from 100 to 5000 so that per-request cost is observed as the ledger and the provenance index grow rather than at a single operating point. ZK-Redact is plotted at redaction batch sizes  $B_R \in \{1, 8, 64\}$  from the full sweep  $B_R \in \{1, 2, 4, 8, 16, 32, 64\}$ , with the waiting bound  $\Delta_R$  swept over  $\{63, 125, 250, 500\}$  ms—a

Fig. 4: Experiment 2: average processing overhead per request versus number of redaction requests. ZK-Redact is shown at redaction batch sizes  $B_R \in \{1, 8, 64\}$ , where  $B_R = 1$  is both the batching-disabled ablation and the operating point of the three baselines.

quarter of a block interval to two block intervals, so that the sweep crosses the point at which a batch spans multiple blocks. The point  $B_R = 1$  is the batching-disabled ablation and also the operating point of all three baselines, none of which batches; the vertical separation between the  $B_R$  curves is therefore the measured amortization, obtained without requiring the baselines to implement a mechanism their designs do not define. Conflict ratio and offered concurrency are swept as described in Section V-B.

*Metrics.* Average processing overhead per request is the primary metric, plotted in Fig. 4. Cost is decomposed rather than aggregated in Table ??, since a single timing cannot identify which component improved and would permit the misreading that batching accelerates redaction cryptography, which the scheme does not claim. The table reports the total CH.Adapt time, which is per-request and therefore the irreducible floor; the blockchain-side time per batch, approximately constant in  $B_R$  and the only part batching can amortize; the achieved redactions per second; and the stale-exclusion rate—the fraction of batched requests for which  $\text{Fresh}_\rho = 0$  and which must be re-authorized. The last is the price of amortization, rises with both  $B_R$  and  $\Delta_R$ , and is given for every plotted configuration so that no reduction in per-request cost is read as free.

*Results.* [To be completed from the measurement run. Report the per-request overhead of each baseline and of ZK-Redact at each batch size, the rising share of the remaining cost that is per-request CH.Adapt cryptography, the diminishing return between  $B_R = 8$  and  $B_R = 64$  as cost approaches the floor, the growth of the stale-exclusion rate, and the resulting operating point at which the marginal amortization gain equals the marginal staleness loss—an operating point that [13] proposes as delayed redaction but never measures.]

**Experiment 3: Provenance Retrieval and Audit Verification Cost: Objective.** Establish whether audit cost scales with the redaction history of the audited transaction rather than with the size of the ledger, which is the property claimed for the PAI in Phase 6.

*Configuration.* The history depth  $\nu \in \{1, 2, 4, 8, 16\}$ —the number of redactions recorded on the audited transaction—is the primary axis. Because the claim concerns two variables at once, each scheme is measured at ledger sizes  $L = 100$  and  $L = 1000$  blocks and both are plotted: the separation between a scheme’s own pair of curves is its measured ledger dependence, and the slope of each curve its history dependence. The number of anchored PAI states follows the redaction rounds executed. Each system performs the audit its design specifies: ZK-Redact retrieves one transaction’s authenticated history and verifies it against the anchored root  $R_{\text{PAI}}^{(e)}$ ; [13] runs its challenge–response audit over the blockchain authentication

Fig. 5: Experiment 3: audit verification time versus number of redactions in the audited transaction’s history. Each scheme is shown at two ledger sizes,  $L = 100$  (solid) and  $L = 1000$  (dashed); coincident curves indicate audit cost independent of ledger length.

tree; [10] queries blocks and recomputes the extended Merkle tree; and [22] collects the current versions of all blocks and compares them against the accumulator state.

*Metrics.* Auditor-side verification time is the primary metric, plotted in Fig. 5. Retrieval latency and evidence transferred are recorded separately in Table ?? rather than summed into the plotted quantity, so that a scheme whose verification cost tracks history depth is not confused with one whose retrieval cost tracks ledger size. Auditor authorization cost—signature verification, freshness checking, and evaluation of  $\Pi_a^{\text{audit}}$ —is reported there as a constant adder, because no prior scheme includes this step and charging it inside the comparison would penalize ZK-Redact for a capability the baselines lack. The schemes answer different audit questions, so what is compared is how cost *scales*, not feature equivalence.

*Results.* [To be completed from the measurement run. Report the growth of verification cost in  $\nu$  for each scheme—ZK-Redact expected to follow  $O(\nu + \log |\mathcal{I}|)$ —together with the separation between each scheme’s  $L = 100$  and  $L = 1000$  curves, expected to be negligible for ZK-Redact, small for [13], and approximately proportional to  $L$  for [10] and [22] since both traverse the chain. State the retrieval latency, evidence size, and constant auditor authorization adder from Table ??.]

## VI. CONCLUSION

### REFERENCES

- [1] M. Jia, J. Chen, K. He, R. Du, L. Zheng, M. Lai, D. Wang, and F. Liu, “Redactable Blockchain from Decentralized Chameleon Hash Functions,” *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2771–2783, 2022, doi: 10.1109/TIFS.2022.3192716.
- [2] K. Huang, X. Zhang, Y. Mu, X. Wang, G. Yang, X. Du, F. Rezaeibagha, Q. Xia, and M. Guizani, “Building Redactable Consortium Blockchain for Industrial Internet-of-Things,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3670–3679, Jun. 2019, doi: 10.1109/TII.2019.2901011.
- [3] W. Wang, L. Wang, J. Duan, X. Tong, and H. Peng, “Redactable Blockchain Based on Decentralized Trapdoor Verifiable Delay Functions,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7492–7507, 2024, doi: 10.1109/TIFS.2024.3431917.
- [4] H. Guo, W. Gan, M. Zhao, C. Zhang, T. Wu, L. Zhu, and J. Xue, “PriChain: Efficient Privacy-Preserving Fine-Grained Redactable Blockchains in Decentralized Settings,” *Chinese Journal of Electronics*, vol. 34, no. 1, pp. 82–97, 2025, doi: 10.23919/cje.2023.00.305.
- [5] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 717–731, 2024, doi: 10.1016/j.jpdc.2023.3261343.
- [6] X. Wu, X. Du, Q. Yang, N. Wang, and W. Wang, “Redactable consortium blockchain based on verifiable distributed chameleon hash functions,” *J. Parallel Distrib. Comput.*, vol. 183, Art. no. 104777, 2024, doi: 10.1016/j.jpdc.2023.104777.
- [7] S. Aguincha, E. Nunes, S. Eisa, and M. L. Pardal, “ChainGuards: Verification of Sensed Data using Permissioned Blockchain Technology,” arXiv preprint arXiv:2603.20769, 2026.
- [8] D. Derler, K. Samelin, D. Slamanig, and C. Striecks, “Fine-Grained and Controlled Rewriting in Blockchains: Chameleon-Hashing Gone Attribute-Based,” in *Proc. 26th Annu. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019, doi: 10.14722/ndss.2019.23066.

- [9] G. Ateniese, B. Magri, D. Venturi, and E. R. Andrade, "Redactable Blockchain – or – Rewriting History in Bitcoin and Friends," in *Proc. 2nd IEEE Eur. Symp. Security Privacy (EuroS&P)*, Paris, France, pp. 111–126, 2017, doi: 10.1109/EuroSP.2017.37.
- [10] Z. Wu, L. Wang, X. Zhang and X. Feng, "EMT: Extended Merkle Tree Structure for Inserted Data Redaction in Permissioned Blockchain," in *IEEE Transactions on Network Science and Engineering*, vol. 12, no. 4, pp. 3025–3038, July-Aug. 2025, doi: 10.1109/TNSE.2025.3555979.
- [11] T. Sengupta, S. Chakraborty and S. Sural, "ReAcct: Redaction Control for Interoperable Blockchains," 2025 7th Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), Zurich, Switzerland, 2025, pp. 1–10, doi: 10.1109/BRAINS67003.2025.11302947.
- [12] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, "Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting," in *Proc. 19th ACM Asia Conf. Computer and Communications Security (AsiaCCS)*, pp. 1076–1091, 2024, doi: 10.1145/3634737.3656291.
- [13] G. Tian, J. Wei, M. Kutyłowski, W. Susilo, X. Huang, and X. Chen, "VRBC: A Verifiable Redactable Blockchain with Efficient Query and Integrity Auditing," *IEEE Trans. Comput.*, vol. 72, no. 7, pp. 1928–1942, Jul. 2023, doi: 10.1109/TC.2022.3230900.
- [14] K. Huang, X. Zhang, Y. Mu, F. Rezaeibagha, and X. Du, "Scalable and redactable blockchain with update and anonymity," *Information Sciences*, vol. 546, pp. 25–41, 2021, doi: 10.1016/j.ins.2020.07.016.
- [15] Y. Dong, Y. Li, Y. Cheng, and D. Yu, "Redactable consortium blockchain with access control: Leveraging chameleon hash and multi-authority attribute-based encryption," *High-Confidence Computing*, vol. 4, no. 1, Art. no. 100168, 2024, doi: 10.1016/j.hcc.2023.100168.
- [16] Y. Zhang, Z. Ma, S. Luo, and P. Duan, "Dynamic Trust-Based Redactable Blockchain Supporting Update and Traceability," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 821–834, 2024, doi: 10.1109/TIFS.2023.3326379.
- [17] F. Wang, R. Dong, J. Cui, Q. Zhang and H. Zhong, "Blockchain-Based Anonymous and Accountable Data Redaction for the Industrial Internet of Things," in *IEEE Transactions on Cloud Computing*, doi: 10.1109/TCC.2026.3725253.
- [18] J. B. Klamti and M. A. Hasan, "Revocable policy-based chameleon hash using lattices," *Journal of Mathematical Cryptology*, vol. 18, no. 1, Art. no. 20230012, pp. 152–182, 2024, doi: 10.1515/jmc-2023-0012.
- [19] X. Huang, Y. Wang, Y. Ding, Q. Wu, C. Yang, and H. Liang, "Dynamically redactable Blockchain based on decentralized Chameleon hash," *Digital Communications and Networks*, vol. 11, no. 3, pp. 757–767, 2025, doi: 10.1016/j.dcan.2024.10.013.
- [20] M. Miao, X. Yang, J. Wei, G. Tian, and W. Susilo, "A Verifiable and Redactable Blockchain with Lightweight Storage and Permission Supervision," *Information*, vol. 17, no. 2, Art. no. 176, 2026, doi: 10.3390/info17020176.
- [21] S. Fugkeaw, S. Sungchai, S. Nakprame, and P. Sreekongpan, "Enabling Secure and Scalable GDPR-Compliant Blockchain-Based e-KYC With Efficient Redaction," *IEEE Access*, vol. 13, pp. 136834–136853, 2025, doi: 10.1109/ACCESS.2025.3594656.
- [22] Y. Shen, et al., "Verifiable and Redactable Blockchains With Fully Editing Operations," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 3792–3805, 2023.